

OSSP (Operating Systems And Systems Programming) - 25CS2104E

SKILL PROGRAMS

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

```
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_1/src$ nano skill1.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_1/src$ gcc skill1.c -o skill1
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_1/src$ ./skill1
```

```
=====
                OSSP - SKILL 1
                LINUX COMMAND EXECUTION
=====

Enter Linux command: ls

[ PARENT PROCESS ]
Parent PID : 650
Child PID  : 653
Status     : Waiting for child...

[ CHILD PROCESS ]
Child PID  : 653
Parent PID : 650
Command    : ls
Status     : Executing command...
skill1    skill1.c

[ PARENT PROCESS ]
Child PID  : 653
Status     : Child execution completed.
Exit Status: 0

=====
                SKILL 1 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_1/src$ |
```

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_2$ nano skill2.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_2$ gcc skill2.c -o skill2
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_2$ ./skill2

=====
      OSSP - SKILL 2
      INTERACTIVE SHELL LOOP
=====

Type 'exit' to quit.

OSSP-Shell> ls

[COMMAND INPUT]
Command : ls
Status  : Command received successfully.

OSSP-Shell> pwd

[COMMAND INPUT]
Command : pwd
Status  : Command received successfully.

OSSP-Shell> hello

[COMMAND INPUT]
Command : hello
Status  : Command received successfully.

OSSP-Shell> exit

[COMMAND INPUT]
Command : exit
Status  : Exit command received.

=====
      SKILL 2 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_2$ |

```

3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_3$ nano skill3.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_3$ gcc skill3.c -o skill3
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_3$ ./skill3
```

```
=====
OSSP - SKILL 3
COMMAND HISTORY & MEMORY
=====

Enter commands.
Type 'history' to display history.
Type 'exit' to finish.

OSSP-Shell> ls

[COMMAND STORED]
Command : ls
Status : Stored in dynamic history list.

OSSP-Shell> pwd

[COMMAND STORED]
Command : pwd
Status : Stored in dynamic history list.

OSSP-Shell> gcc

[COMMAND STORED]
Command : gcc
Status : Stored in dynamic history list.

OSSP-Shell> history

=====
COMMAND HISTORY
=====
1. ls
2. pwd
3. gcc
=====

OSSP-Shell> exit

Exit command received.

=====
SKILL 3 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_3$ |
```

-
4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_4$ nano skill4.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_4$ gcc skill4.c -o skill4
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_4$ ./skill4

=====
OSSP - SKILL 4
TOKENIZATION & PARSER
=====

Enter a command to tokenize.
Type 'exit' to quit.

OSSP-Parser> ls -l

=====
TOKEN STREAM
=====
Token 1 : ls
Token 2 : -l
=====

=====
PARSE STRUCTURE
=====
COMMAND
|
+-- PROGRAM: ls
+-- ARGUMENT: -l
=====

OSSP-Parser> exit

Exit command received.

=====
SKILL 4 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_4$ |

```

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ nano skill5.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ gcc skill5.c -o skill5
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ ./skill5

=====
O SSP - SKILL 5
QUOTING AND EXPANSION
=====

Single quotes preserve literal content.
Double quotes preserve spaces and allow variable expansion.
Type 'exit' to quit.

OSSP-Quotes> 'Hello World'

[ SINGLE QUOTE PROCESSING ]
Input : 'Hello World'
Rule : Content inside single quotes is treated literally.
Result: Hello World

OSSP-Quotes> "Hello World"

[ DOUBLE QUOTE PROCESSING ]
Input : "Hello World"
Rule : Spaces are preserved and variables can be expanded.
Result: Hello World

OSSP-Quotes> exit

Exit command received.

=====
SKILL 5 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ |

```

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ gcc skill6.c -o skill6
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ ./skill6

=====
OSSP - SKILL 6
ESCAPES & EXECUTION
=====

Examples:
ls -l
echo Hello\ World
echo Hello\!

Type 'exit' to quit.

OSSP-Shell> ls

[ PARSED COMMAND ]
Argument 1 : ls

[ PARENT PROCESS ]
PID       : 1201
Child PID  : 1202
Status    : Waiting for child...

[ CHILD PROCESS ]
PID       : 1202
Parent PID : 1201
Executing  : ls

docs results skill5 skill5.c skill6 skill6.c src

Child exited normally.
Exit status: 0

OSSP-Shell> exit

Exit command received.

=====
SKILL 6 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ |

```

7. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ nano skill7.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ gcc skill7.c -o skill7
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ ./skill7

=====
      OSSP - SKILL 7
      WAITPID & PATH RESOLUTION
=====

This program searches PATH for commands
and synchronizes with child processes.
Type 'exit' to quit.

OSSP-Shell> ls

[ COMMAND RESOLUTION ]
Command   : ls
Executable : /usr/bin/ls

[ PARENT PROCESS ]
Parent PID : 1281
Child PID  : 1284
Waiting using waitpid()...

[ CHILD PROCESS ]
Child PID  : 1284
Parent PID : 1281
docs results skills5 skill5.c skill6 skill6.c skill7 skill7.c src

[ CHILD PROCESS STATUS ]
Child exited normally.
Exit status: 0

OSSP-Shell> exit

Exit command received.

=====
      SKILL 7 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ |

```

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ nano skill8.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ gcc skill8.c -o skill8
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ ./skill8

=====
      OSSP - SKILL 8
      VARIABLE & BUILT-IN DISPATCH
=====

Supported built-ins:
  pwd
  echo
  exit

Variable example:
  $HOME
  $USER
  $PATH

OSSP-Shell> $HOME

[VARIABLE EXPANSION]
Input : $HOME
Value : /home/nikhilreddy
Status: Variable expanded successfully.
Invalid built-in command: /home/nikhilreddy

OSSP-Shell> $USER

[VARIABLE EXPANSION]
Input : $USER
Value : nikhilreddy
Status: Variable expanded successfully.
Invalid built-in command: nikhilreddy

OSSP-Shell> exit

Exit command received.

=====
      SKILL 8 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ |

```

-
9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ nano skill9.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ gcc skill9.c -o skill9
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ ./skill9

=====
                OSSP - SKILL 9
                DIRECTORY & BUILT-IN
=====

Supported commands:
  pwd
  cd <directory>
  cd -
  exit
[PWD]
Current directory : /home/nikhilreddy/OSSP/Projects/OSSP_Skill_5

OSSP-Shell> cd /tmp
[CD]
Previous directory : /home/nikhilreddy/OSSP/Projects/OSSP_Skill_5
Current directory : [PWD]
Current directory : /tmp
Status : Directory changed successfully.

OSSP-Shell> pwd
[PWD]
Current directory : /tmp

OSSP-Shell> exit

Exit command received.

=====
                SKILL 9 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_5$ |

```

-
10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill10.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill10.c -o skill10
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill10

=====
                OSSP - SKILL 10
                MINI SHELL ENVIRONMENT
=====

Supported commands:
  pwd
  export NAME=VALUE
  env
  echo $NAME
  exit

OSSP-SKILL10:/home/nikhilreddy/OSSP/Projects/OSSP_Skill_10$ pwd
[PWD]
Current directory : /home/nikhilreddy/OSSP/Projects/OSSP_Skill_10

OSSP-SKILL10:/home/nikhilreddy/OSSP/Projects/OSSP_Skill_10$ export STUDENT=NIKHIL

[EXPORT]
Variable : STUDENT
Value    : NIKHIL
Status   : Environment variable exported successfully.

OSSP-SKILL10:/home/nikhilreddy/OSSP/Projects/OSSP_Skill_10$ |

```

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill11.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill11.c -o skill11
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill11

=====
OSSP - SKILL 11
COMMAND HISTORY
=====

History capacity : 5 commands
Type 'history' to display history.
Type 'exit' to quit.

OSSP-SKILL11$ ls

[STORED]
Command : ls
Status : Command added to history.

OSSP-SKILL11$ pwd

[STORED]
Command : pwd
Status : Command added to history.

OSSP-SKILL11$ whoami

[STORED]
Command : whoami
Status : Command added to history.

OSSP-SKILL11$ history

=====
COMMAND HISTORY
=====
1  ls
2  pwd
3  whoami
=====

OSSP-SKILL11$ exit

=====
SKILL 11 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |
```

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill12.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill12.c -o skill12
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill12

=====
OSSP - SKILL 12
PIPE DEMO
=====

[PIPE]
Pipe created successfully.
Read descriptor : 3
Write descriptor : 4

[PARENT PROCESS]
PID : 639
Child PID : 640
Message to pipe: Message sent from parent process
Data written to pipe successfully.

Waiting for child...
[CHILD PROCESS]
PID : 640
Parent PID : 639
Data received from pipe:
Message sent from parent process
Child received data successfully.

[SYNCHRONIZATION]
Parent waited for child successfully.
Child exit status: 0

=====
SKILL 12 DEMO COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |

```

-
13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill13.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill13.c -o skill13
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill13

=====
OSSP - SKILL 13
INPUT / OUTPUT REDIRECTION
=====

Commands:
input <filename>
output <filename> <text>
exit

OSSP-Redirect> pwd
Unknown command: pwd

OSSP-Redirect> output output.txt Hello_World

[OUTPUT REDIRECTION]
Text : Hello_World
File : output.txt
stdout : redirected successfully

OSSP-Redirect> exit

=====
SKILL 13 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |

```

-
14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill14.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill14.c -o skill14
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill14

=====
OSSP - SKILL 14
APPEND & STDERR REDIRECTION
=====

Commands:
  append <file> <text>
  error <file>
  exit

OSSP-Redirect> append data.txt First_Line

[APPEND REDIRECTION]
Operator : >>
File      : data.txt
Text      : First_Line
Status    : Data appended successfully.

OSSP-Redirect> append data.txt Second_Line

[APPEND REDIRECTION]
Operator : >>
File      : data.txt
Text      : Second_Line
Status    : Data appended successfully.

OSSP-Redirect> append data.txt Third_Line

[APPEND REDIRECTION]
Operator : >>
File      : data.txt
Text      : Third_Line
Status    : Data appended successfully.

OSSP-Redirect> exit

=====
SKILL 14 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |

```

-
15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill15.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill15.c -o skill15
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill15

=====
OSSP - SKILL 15
COMBINED REDIRECTION
=====

Commands:
  combine <file>
  separate <out> <err>
  exit

OSSP-Complex> combine combined.txt

[COMBINED REDIRECTION]
stdout    -> combined.txt
stderr    -> combined.txt
Status    : Both streams merged successfully.

OSSP-Complex> exit

=====
SKILL 15 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |

```

-
16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill16.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill16.c -o skill16
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill16

=====
      OSSP - SKILL 16
      BACKGROUND JOBS
=====

Commands:
<command> &      Run command in background
jobs             Show active jobs
check            Check completed jobs
exit             Exit shell

OSSP-Jobs> sleep 10 &
[Job 1] Started background process PID 878

OSSP-Jobs> jobs

===== BACKGROUND JOBS =====
[1] PID=878 RUNNING  sleep 10
=====

OSSP-Jobs> check

[Job 1] PID 878 completed with exit status 0

OSSP-Jobs> exit

=====
      SKILL 16 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |

```

-
17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```

nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill17.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill17.c -o skill17
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill17

=====
      OSSP - SKILL 17
      JOBS AND FOREGROUND CONTROL
=====

Commands:
<command> &      Run in background
jobs             Display jobs
fg <job_id>      Bring job to foreground
check            Check completed jobs
exit             Exit shell

OSSP-FG> sleep 20 &
[Job 1] Started PID 928

OSSP-FG> jobs

===== JOB LIST =====
[1] PID=928 RUNNING  sleep 20
=====

OSSP-FG> fg 1
Bringing Job 1 (PID 928) to foreground...
Job 1 completed.

OSSP-FG> exit

=====
      SKILL 17 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ |

```

-
18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ nano skill18.c
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ gcc skill18.c -o skill18
nikhilreddy@Hp:~/OSSP/Projects/OSSP_Skill_10$ ./skill18
```

OSSP - SKILL 18
SIGNALS

Process ID (PID): 970

```
Signal handlers installed.
```

Available tests:

```
Ctrl+C      -> SIGINT
kill -TERM PID -> SIGTERM
kill -USR1 PID -> SIGUSR1
```

```
Process is waiting for signals...
```

[illegible]

```
^C
[Signal Handler] SIGINT received.
```

Process is still running.

[illegible]

```
Process running... PID = 970
Process running... PID = 970
^C
```

19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```

nikhilreddy@Hp:~/OSSP/Projects$ nano skill19.c
nikhilreddy@Hp:~/OSSP/Projects$ gcc skill19.c -o skill19
nikhilreddy@Hp:~/OSSP/Projects$ ./skill19

=====
OSSP - SKILL 19
SIGNAL HANDLER - SIGINT
=====

Process ID (PID): 1100

SIGINT handler installed successfully.
Press Ctrl+C to send SIGINT.
Press Ctrl+\ to terminate the program.

Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
^C
[Signal Handler] SIGINT received!
Interrupt count: 1
Press Ctrl+C again to test the handler.
Program running... PID = 1100
Program running... PID = 1100
^C
[Signal Handler] SIGINT received!
Interrupt count: 2
Press Ctrl+C again to test the handler.
Program running... PID = 1100
Program running... PID = 1100
Program running... PID = 1100
^\\Quit      ./skill19
nikhilreddy@Hp:~/OSSP/Projects$ |

```

-
20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```

nikhilreddy@Hp:~/OSSP/Projects$ nano skill20.c
nikhilreddy@Hp:~/OSSP/Projects$ gcc skill20.c -o skill20
nikhilreddy@Hp:~/OSSP/Projects$ ./skill20

=====
OSSP - SKILL 20
SIGTSTP & PROCESS GROUPS
=====

Process ID      : 1147
Parent Process ID: 993
Process Group ID : 1147

SIGTSTP handler installed.
Press Ctrl+Z to send SIGTSTP.
Press Ctrl+C to send SIGINT.
Use Ctrl+\\ to terminate.

Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
Process running... PID=1147  PGID=1147
^Z
[Signal Handler] SIGTSTP received!
SIGTSTP count: 1
Process group ID: 1147
Process continues running.
Process running... PID=1147  PGID=1147
^C
nikhilreddy@Hp:~/OSSP/Projects$ |

```

-
21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```

nikhilreddy@Hp:~/OSSP/Projects$ nano skill21.c
nikhilreddy@Hp:~/OSSP/Projects$ gcc skill21.c -o skill21
nikhilreddy@Hp:~/OSSP/Projects$ ./skill21

=====
      OSSP - SKILL 21
    PROCESS GROUPS & TERMINAL CONTROL
=====

[PARENT BEFORE FORK]

===== PROCESS INFORMATION =====
Process ID (PID)      : 1198
Parent Process ID     : 993
Process Group ID      : 1198
Session ID            : 993
Terminal Process Group : 1198
=====

[PARENT PROCESS]
Parent PID      : 1198
Parent PGID     : 1198
Child PID       : 1199

Waiting for child process...

[CHILD PROCESS]
Child PID       : 1199
Parent PID      : 1198
Child PGID      : 1199
Child Session ID : 993
Terminal PGID   : 1198

Child process group created successfully.
Child process will run for 5 seconds...
Child process completed.
Child exited normally.
Child exit status: 0

=====
      SKILL 21 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects$ |

```

22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
nikhilreddy@Hp:~/OSSP/Projects$ nano skill22.c
nikhilreddy@Hp:~/OSSP/Projects$ gcc skill22.c -o skill22
nikhilreddy@Hp:~/OSSP/Projects$ ./skill22

=====
OSSP - SKILL 22
VALGRIND & TESTING
=====

Enter a message: OSSP Testing

===== TEST RESULT =====
Input message : OSSP Testing
Length       : 12
Memory       : allocated successfully
=====

Memory released successfully.

=====
SKILL 22 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects$ |
```

-
23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

```
nikhilreddy@Hp:~/OSSP/Projects$ nano skill23.c
nikhilreddy@Hp:~/OSSP/Projects$ gcc skill23.c -o skill23
nikhilreddy@Hp:~/OSSP/Projects$ ./skill23

=====
OSSP - SKILL 23
LARGE PIPELINE & PERFORMANCE
=====
3

=====
PIPELINE COMPLETED
=====

Pipeline executed:
printf -> tr -> sort -> uniq -> wc

Processes created: 5
Pipes created    : 4

Performance concept demonstrated:
Multiple processes execute concurrently
while data flows through UNIX pipes.

=====
SKILL 23 EXECUTION COMPLETED
=====
nikhilreddy@Hp:~/OSSP/Projects$ |
```

-
24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

```
nikhilreddy@Hp:~/OSSP/Projects$ nano skill 24.c
nikhilreddy@Hp:~/OSSP/Projects$ nano skill24.c
nikhilreddy@Hp:~/OSSP/Projects$ gcc skill24.c -o skill24
nikhilreddy@Hp:~/OSSP/Projects$ ./skill24
```

```
=====
OSSP - SKILL 24
FINAL DEMONSTRATION & REVIEW
=====
```

```
===== OSSP FINAL MENU =====
```

1. System Information
2. Process Demonstration
3. Pipe Demonstration
4. Run Complete Demonstration
5. Exit

```
=====
Enter your choice: 1
```

```
===== SYSTEM INFORMATION =====
```

```
Operating System : Linux/Ubuntu
Process ID       : 1359
Parent Process   : 993
Process Group ID : 1359
=====
```

```
===== OSSP FINAL MENU =====
```

1. System Information
2. Process Demonstration
3. Pipe Demonstration
4. Run Complete Demonstration
5. Exit

```
=====
Enter your choice: 2
```

```
===== PROCESS DEMONSTRATION =====
```

```
Parent Process
Parent PID : 1359
Child PID  : 1360
Child Process
Child PID  : 1360
Parent PID : 1359
Child process completed.
=====
```

```
===== OSSP FINAL MENU =====
```

1. System Information
2. Process Demonstration
3. Pipe Demonstration
4. Run Complete Demonstration
5. Exit

```
=====
Enter your choice: 5
```

```
Exiting Skill 24...
OSSP FINAL REVIEW COMPLETED.
```